

Product Requirements Document (PRD)

MedFlow — Autonomous Healthcare Operations Suite

Version: 1.0 **Author:** [Your Name] **Date:** August 2026 **Domain:** Healthcare Information Technology / Intelligent Process Automation **Tech Stack:** Python, Supabase (PostgreSQL + Auth + Storage), FastAPI, React (optional frontend)

1. Overview

1.1 Purpose

MedFlow is an intelligent automation platform designed to unify and streamline core hospital operations — administrative, clinical, billing, pharmacy, and compliance — into a single system. It replaces fragmented manual processes with automated workflows, anomaly detection, and smart notifications, reducing operational overhead and human error in healthcare settings.

1.2 Problem Statement

Hospitals and clinics rely on disconnected manual or semi-manual processes for scheduling, documentation, billing, pharmacy inventory, and compliance tracking. This leads to:

Missed appointments and poor patient follow-up

Billing leakage and claim denials

Medication stock-outs or expiries

Compliance/documentation gaps

Delayed escalation of urgent patient issues

1.3 Goal

Build a modular, automation-first platform (MedFlow) that connects these workflows through a shared Supabase-backed data layer, with Python-based automation agents handling each domain.

1.4 Target Users

Hospital administrative staff

Nurses and clinical staff

Billing/claims department

Pharmacy staff

Compliance officers

(Simulated/mock data for internship-scope demo; not for production clinical use)

2. Scope (Internship-Realistic MVP)

Given internship timelines, MedFlow will be built as a **modular MVP** — a working core platform with 2-3 fully functional modules, and 2-3 partially simulated/prototype modules to demonstrate the full vision.

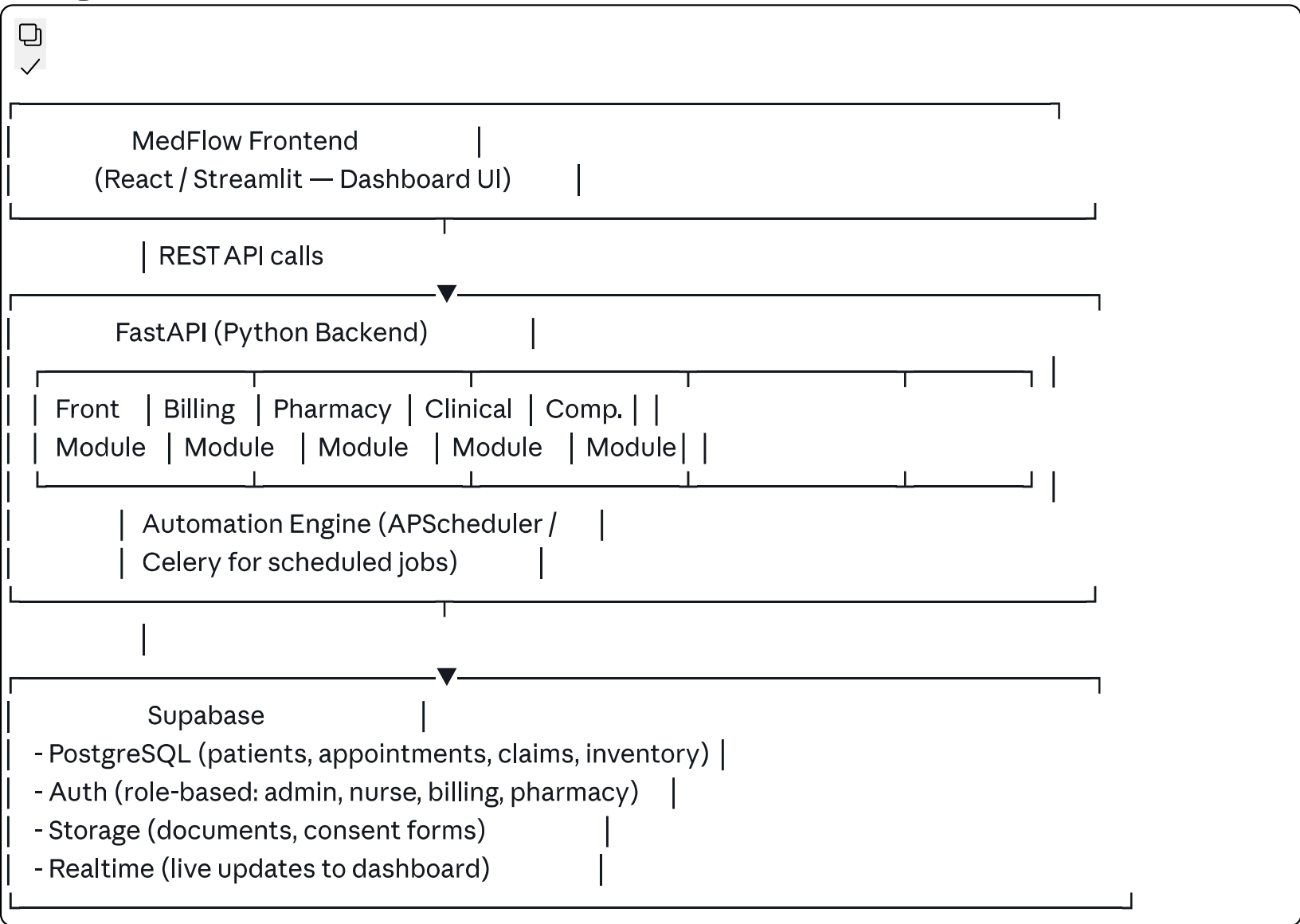
Recommended MVP priority:

- ✓ MedFlow Front (Scheduling + Intake) — full build
- ✓ MedFlow Billing (Claims + Ghost-charge detection) — full build
- ✓ MedFlow Pharmacy (Inventory automation) — full build

- ◆ MedFlow Clinical (Documentation automation) — partial/prototype
- ◆ MedFlow Compliance (Audit bot) — partial/prototype
- ◆ MedFlow Insight (Anomaly detection) — partial/prototype

3. System Architecture

3.1 High-Level Architecture



3.2 Tech Stack Breakdown

Layer	Technology
Backend	Python 3.11+, FastAPI
Database	Supabase (PostgreSQL)
Auth	Supabase Auth (role-based access)
Task Scheduling	APScheduler or Celery + Redis
Notifications	Twilio (SMS) / SendGrid (Email) — sandbox/test mode
OCR (for intake/consent docs)	Tesseract / pytesseract
Frontend	Streamlit (fast MVP) or React (if time allows)
Hosting	Render / Railway (backend), Supabase (managed DB)

4. Functional Requirements by Module

4.1 MedFlow Front (Scheduling & Intake)

FR1: Patients can book/cancel/reschedule appointments via API

FR2: Automated SMS/email reminders sent 24h and 2h before appointment

FR3: No-show detection → auto-trigger follow-up/reschedule workflow

FR4: Insurance eligibility check (mocked API/rules-based simulation)

FR5: Digital intake form → OCR extraction → structured DB entry

4.2 MedFlow Billing (Claims Automation)

FR6: Auto-generate claim record from completed appointment + service codes

FR7: Claim status tracking (submitted, pending, denied, paid)

FR8: Denial detection → auto-flag with reason code + draft resubmission note

FR9: **Ghost-charge detector**: rule-based cross-check that billed procedure/supply matches patient's diagnosis/age group/department; flag mismatches

4.3 MedFlow Pharmacy (Inventory Automation)

FR10: Track medication stock levels in real time

FR11: Auto-trigger reorder request when stock < threshold

FR12: Expiry date monitoring → auto-alert X days before expiry

FR13: Prescription refill request → auto-queue to pharmacy dashboard

4.4 MedFlow Clinical (Documentation) — Prototype

FR14: Auto-generate discharge summary template from structured visit data

FR15: Auto-flag incomplete patient records (missing required fields)

4.5 MedFlow Compliance (Audit) — Prototype

FR16: Automated checklist scan of documents for required consent fields

FR17: Consent contradiction checker (e.g., allergy mismatch across forms)

4.6 MedFlow Insight (Anomaly Detection) — Prototype

FR18: Referral leakage tracker — flags referrals with no completed visit after X days

FR19: Medication adherence pattern flagging (missed refill patterns)

5. Non-Functional Requirements

Category	Requirement
Security	Role-based access control via Supabase Auth; no real PHI — use synthetic/mock data only
Performance	API responses < 500ms for standard queries
Scalability	Modular service design — each module independently deployable
Compliance	Simulate HIPAA-aware design patterns (data minimization, audit logs) — not a certified compliant system
Reliability	Scheduled jobs (reminders, reorder checks) run via cron/APScheduler with logging
Auditability	All automation actions logged with timestamp + triggering rule

6. Data Model (Core Supabase Tables)

✓ patients (id, name, dob, contact, allergies, insurance_id) appointments (id, patient_id, provider_id, datetime, status, no_show_flag) claims (id, appointment_id, procedure_code, amount, status, denial_reason) inventory (id, medication_name, stock_qty, threshold, expiry_date) prescriptions (id, patient_id, medication_id, refill_status) consent_forms (id, patient_id, form_type, content_hash, submitted_at) audit_log (id, module, action, triggered_by_rule, timestamp) referrals (id, patient_id, referred_to, referral_date, completed_flag)

7. Automation Logic Examples (Python)

Example: No-show → auto-reschedule trigger

✓ python
--

```
def check_no_shows():
    overdue = supabase.table("appointments") \
        .select("*") \
        .eq("status", "no_show") \
        .execute()
    for appt in overdue.data:
        trigger_reschedule_workflow(appt["patient_id"])
        log_action("MedFlow Front", "auto_reschedule_triggered", appt["id"])
```

Example: Ghost-charge detection (simplified rule)




```
python
def detect_ghost_charges(claim):
    if claim["procedure_category"] == "pediatric" and claim["patient_age"] > 18:
        flag_claim(claim["id"], reason="Age-procedure mismatch")
```

8. Milestones / Phased Plan (Suggested 8-10 Week Internship Timeline)

Phase	Duration	Deliverable
Phase 1: Setup	Week 1	Supabase schema, FastAPI skeleton, auth setup
Phase 2: MedFlow Front	Week 2-3	Scheduling + reminders + no-show automation
Phase 3: MedFlow Billing	Week 4-5	Claims pipeline + ghost-charge detector
Phase 4: MedFlow Pharmacy	Week 6	Inventory automation + reorder triggers
Phase 5: Prototype Modules	Week 7	Clinical + Compliance + Insight (basic versions)
Phase 6: Dashboard + Polish	Week 8	Streamlit/React dashboard, demo data, final report

9. Success Metrics (for demo/evaluation)

% reduction in simulated no-show rate after automated reminders

Number of ghost-charges correctly flagged in test dataset

Time saved (simulated) per automated workflow vs. manual process

Number of compliance/consent issues caught by audit bot

10. Out of Scope (Important for Internship Framing)

Real patient data / production HIPAA compliance certification

Integration with live hospital EHR systems (e.g., Epic, Cerner)
Real insurance API integration (will use mocked/simulated logic)
Clinical decision-making or diagnostic recommendations

11. Risks & Assumptions

Assumption: All data used will be synthetic/mock (e.g., via Faker library) to avoid PHI/privacy issues

Risk: Scope creep across 6 modules — mitigate by strictly following MVP-first phasing

Risk: Supabase free-tier limits (rows, storage) — monitor usage during testing

This PRD is scoped for an academic/internship-level implementation and demo, not a production healthcare deployment.